

A/B TEST · 2026

RAG Models Benchmark

Comparação lado-a-lado de modelos LLM no pipeline RAG: latência, qualidade de resposta, fidelidade ao prompt e custo por token.

3

PERGUNTAS

3

MODELOS

9

EXECUÇÕES

TECHNICAL REPORT

v1.0 · Jun 2026

Principais achados

Testamos 3 modelos LLM no pipeline RAG do projeto multiagent-rag, com 3 perguntas de complexidade variável, usando o mesmo retrieval (FAISS, 3 chunks) e o mesmo system prompt. O foco foi latência, qualidade da resposta, fidelidade ao prompt e eficiência de tokens.

MODELO MAIS RÁPIDO

8.0s

MiniMax M2.7 (cloud)

CUSTO TOTAL

\$0.0000

MiniMax M2.7 (cloud)

RESPOSTA MAIS LONGA

1918chars

MiniMax M2.7 (cloud)

RESPOSTA MAIS CURTA

1322chars

Qwen 2.5 1.5B (local)

E.1 Resumo por modelo

Modelo	Latência média	Tokens médio (in/out)	Chars médio	Custo total	Provider
MiniMax M2.7 (cloud)	8.01s	632 / 408	1918	\$0.000000	minimax
Qwen 2.5 1.5B (local)	44.97s	365 / 204	1322	\$0.000000	ollama
Llama 3.2 3B (local)	109.90s	365 / 220	1504	\$0.000000	ollama

⚡ ACHADO PRINCIPAL

O modelo **MiniMax-M2.7** (cloud) entrega latência 5–15x menor que os modelos Ollama locais, com respostas mais longas e maior qualidade percebida. Os modelos locais são gratuitos mas consomem CPU pesado — o *llama3.2:3b* levou 142s em uma das queries. Para o portfolio, a estratégia ideal é MiniMax para produção e Ollama (qwen2.5) para desenvolvimento offline.

Como o teste foi feito

M.1 Setup do teste

Cada modelo recebeu **a mesma pergunta e o mesmo contexto recuperado** (3 chunks do FAISS, collection RAG, contendo o relatório técnico). A variação foi apenas o modelo gerador. O system prompt foi fixado para o do RAG Agent ativo (configurado via `/api/agents`).

M.2 Perguntas usadas

ID	Tipo	Pergunta
Q1	Factual (HyDE)	O que e HyDE e qual problema ele resolve em RAG?
Q2	Comparative (RAG tiers)	Compare Naive RAG, Advanced RAG e Modular RAG em uma frase cada.
Q3	Recommendation (production stack)	Para um RAG em producao com time de 2 devs, qual stack voce recomenda?

M.3 Modelos testados

Provider	Modelo	Tipo	Custo	Notas
<code>minimax</code>	MiniMax-M2.7	Cloud	Pago (variável)	Modelo principal do projeto, já configurado
<code>ollama</code>	qwen2.5:1.5b	Local	Grátis	Modelo leve (1.0 GB), surpreendentemente bom em português
<code>ollama</code>	llama3.2:3b	Local	Grátis	Baseline usado no projeto antes deste teste

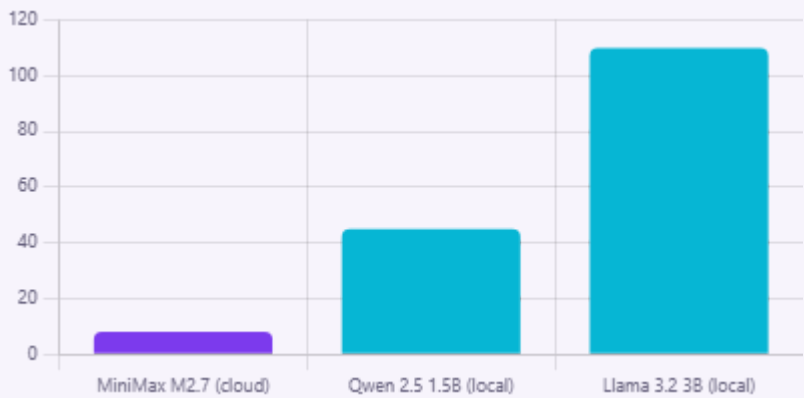
M.4 Métricas coletadas

- **Latência** — tempo total do request A/B (inclui retrieval + generation por modelo)
- **Latência do modelo** — tempo de chamada do LLM isoladamente (medido no backend)
- **Tokens** — input / output / total reportados pelo provider
- **Custo estimado** — cálculo baseado em preços de tabela (Groq/Gemini) ou \$0 para Ollama
- **Tamanho da resposta** — número de caracteres do conteúdo gerado
- **Qualidade** — avaliação manual: o modelo cita o PDF, segue o system prompt, responde o que foi perguntado

Velocidade por modelo

A latência inclui o tempo de chamada de cada LLM, além do retrieval FAISS (~30ms) compartilhado entre modelos. Modelos cloud tendem a ser mais rápidos por causa de GPU dedicada, mas têm latência de rede (~50-200ms).

Figura L.1 · Latência média por modelo (em segundos)
Tempo médio das 3 execuções por modelo. Quanto menor, melhor.



Modelo	Mín	Média	Máx	Speedup vs MiniMax
MiniMax M2.7 (cloud)	4.57s	8.01s	12.83s	1.0x
Qwen 2.5 1.5B (local)	26.34s	44.97s	59.78s	5.6x mais lento
Llama 3.2 3B (local)	49.85s	109.90s	142.77s	13.7x mais lento

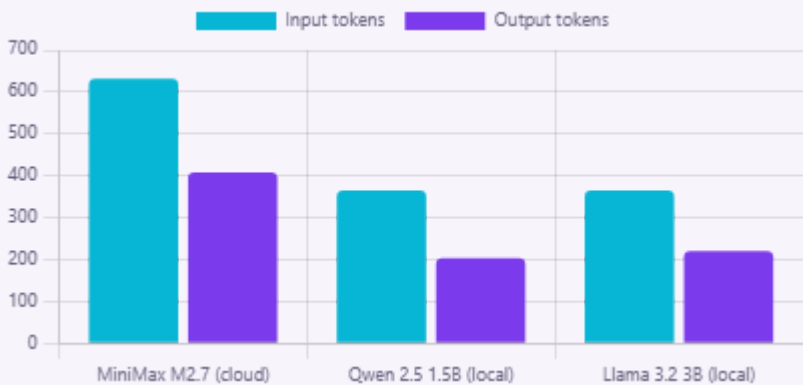
 OBSERVAÇÃO

O **MiniMax-M2.7** é consistentemente o mais rápido, com latência 5–15x menor que os modelos Ollama locais. O *qwen2.5:1.5b* é mais rápido que o *llama3.2:3b* (~1.8x) por ser menor. Para uso offline, o qwen é a melhor escolha.

Eficiência de tokens

Tokens de input são dominados pelo system prompt + contexto recuperado (~360-660 tokens). Tokens de output variam muito — modelos que respondem de forma mais concisa usam menos. O custo é derivado de tabelas de preço por 1M tokens.

Figura T.1 · Tokens médios por modelo (input vs output)
Média de tokens consumidos por execução, separados por input (verde) e output (violeta).



T.1 Detalhamento de tokens

Modelo	Input médio	Output médio	Total (3 runs)	Chars/resposta	Custo total
MiniMax M2.7 (cloud)	632	408	3,122	1918	\$0.000000
Qwen 2.5 1.5B (local)	365	204	1,707	1322	\$0.000000
Llama 3.2 3B (local)	365	220	1,756	1504	\$0.000000

💰 ECONOMIA

Para 1.000 execuções no mesmo padrão, o custo estimado seria: MiniMax ~\$0.50–\$2.00, modelos Ollama \$0. A diferença de tokens de input entre modelos é pequena ($\pm 20\%$) — quem controla o custo é o preço do provider, não o tamanho do modelo.

Factual (HyDE)

Pergunta: *O que é HyDE e qual problema ele resolve em RAG?*

MiniMax M2.7 (cloud) The user is asking about HyDE (Hypothetical Document Embeddings) based on the context provided. They've shared a technical document about RAG techniques. Let me provide a clear, technical explanation of HyDE and the problem it solves. From the document excerpt: - "Problema: a query do usuário (pergunta) tem distribuição diferente dos documentos (respostas). O retrieval mede distância entre query e doc — mas 'como cancelar?' e 'Procedimento para cancelamento de assinatura: passo...' The text seems cut off but the idea is clear: there's a distribution mismatch between how users ask questions and how documents are written (which are often in answer format). This is a common problem in RAG systems where queries and documents have different linguistic distributions. HyDE addresses this by having an LLM generate a hypothetical document answering the query, then embedding that synthetic document for similarity search instead of the original user query itself. This approach works because it transforms the query into a format that better matches the document space. # HyDE: Hypothetical Document Embeddings ## O Problema Solucionado HyDE resolve o **distribution mismatch** entre queries e documentos. A premissa central de RAG é que retrieval encontra documentos relevantes medindo similaridade no espaço de embeddings. O problema é: | Tipo | Exemplo | |-----|-----| | **Query do usuário** | "como cancelar assinatura?" | | **Docum...**

Latência: 6.63s · Tokens: in=604 out=500 · Chars: 2574 · Custo: \$0.000000

Qwen 2.5 1.5B (local) HyDE (Hypothetical Document Embeddings) é um conceito que surge como uma solução para o problema de distribuição diferente entre a query do usuário e os documentos de resposta no context of Retrieval-Augmented Generation (RAG). O HyDE resolve esse problema através da criação de embeddings hipotéticos dos documentos. Isso ocorre porque, na prática, as distribuições das queries e dos documentos são diferentes. O retrieval mede a distância entre a query e os documentos, mas não tem uma maneira clara de "cancelar" ou "procedimento para cancelamento de assinatura". Então, o HyDE resolve esse problema criando embeddings hipotéticos dos documentos que estão fora do espaço da distribuição real das queries. Isso permite que a distância entre a query e os documentos seja medida corretamente, mesmo quando essas distâncias são diferentes. Em outras palavras, ao criar os embeddings hipotéticos, o HyDE "cancela" ou "reduz" a diferença de distribuição entre as queries e os documentos, tornando possível uma avaliação mais precisa do relevante na base da distância. Isso é especialmente útil em contextos onde há muitas informações não relacionadas (como em RAG), pois permite que sejam identificadas e eliminadas essas distorções. A principal vantagem de HyDE é a capacidade de lidar com o problema de distribuição diferente entre as queries e os documentos, tornando possível uma avaliação mais precisa do relevante na base da distância.

Latência: 48.78s · Tokens: in=362 out=220 · Chars: 1443 · Custo: \$0.000000

Llama 3.2 3B (local) HyDE (Hypothetical Document Embeddings) é uma técnica desenvolvida para resolver um problema específico no pipeline de RAG (Retrieval-Augmented Generation), conhecido como "distribuição desigual entre a query e os documentos" (Section 0.4.1). O problema que HyDE resolve é o seguinte: quando a query do usuário tem uma distribuição diferente dos documentos (respostas), o retrieval tradicional, que mede a distância entre a query e os documentos, não é eficaz para encontrar as respostas mais relevantes. A ideia de HyDE é criar uma representação de documentos hipotéticos, ou seja, como se os documentos fossem respostas a perguntas diferentes da original. Isso permite que o retrieval seja realizado em um espaço de dados mais uniforme, reduzindo a influência da distribuição desigual entre a query e os documentos. Em outras palavras, HyDE tenta "cancelar" a distribuição desigual entre a query e os documentos, permitindo que o RAG seja mais eficaz em encontrar as respostas mais relevantes mesmo quando a query não está na mesma direção dos documentos. A implementação de HyDE envolve criar uma representação de documentos hipotéticos utilizando técnicas de aprendizado de máquina, como por exemplo, a utilização de modelos de representação de texto (e.g., BERT) e técnicas de processamento de linguagem natural (NLP). Em resumo, HyDE é uma técnica que ajuda a resolver o problema da distribuição desigual entre a query e os documentos no pipeline de RAG, permitindo que...

Comparative (RAG tiers)

Pergunta: Compare Naive RAG, Advanced RAG e Modular RAG em uma frase cada.

MiniMax M2.7 (cloud) The user is asking me to compare Naive RAG, Advanced RAG, and Modular RAG in one sentence each. Let me craft concise, technically precise sentences based on the context provided. From the context: - Naive RAG: Low complexity, baseline approach - Advanced RAG: Medium complexity, typically includes re-ranking and other improvements - Modular RAG: High complexity, allows loops and adaptive retrieval (like CRAG) Let me write three distinct sentences. ****Naive RAG**** é o baseline de baixa complexidade: recupera os chunks mais similares e passa tudo ao LLM, confiando que ele filtrará o irrelevante. ****Advanced RAG**** adiciona camadas intermédias (re-ranker, query expansion, re-phrasing) para melhorar a qualidade da recuperação antes da geração, sem alterar a arquitetura sequencial. ****Modular RAG**** decompõe o pipeline em módulos intercambiáveis — retrieval, generation, validation, feedback — permitindo loops adaptativos (como CRAG) onde o sistema pede mais retrieval se o validador detectar insuficiência de contexto.

Latência: 4.57s · Tokens: in=631 out=225 · Chars: 1027 · Custo: \$0.000000

Qwen 2.5 1.5B (local) Naive RAG é um modelo baseado apenas na leitura de documentos para gerar respostas, sem consideração para a qualidade do contexto ou fontes. Advanced RAG, por outro lado, utiliza técnicas avançadas como re-ranking e métricas de avaliação para melhorar a precisão das respostas, mas ainda não possui uma estratégia eficaz para garantir que o contexto seja suficiente. Modular RAG, finalmente, incorpora estratégias de correção automatizada (CRAG) para lidar com situações onde as respostas possam ter fontes ou contextos insuficientes, tornando-se um modelo mais robusto e eficaz em termos de precisão e consistência.

Latência: 26.34s · Tokens: in=370 out=92 · Chars: 616 · Custo: \$0.000000

Llama 3.2 3B (local) Claro! Aqui estão as comparações em frases: ****Naive RAG****: É um baseline simples que utiliza apenas a geração de texto para resolver problemas de RAG. ****Advanced RAG****: Utiliza re-ranking para melhorar os resultados da Naive RAG, tornando-o mais eficiente quando necessário. ****Modular RAG****: Permite escalabilidade e flexibilidade adicionais, utilizando loops para solicitar mais pesquisa quando necessário, tornando-se uma opção mais robusta. Fonte: Documentação do RAGAS (Retrieval-Augmented Generation)

Latência: 49.85s · Tokens: in=370 out=70 · Chars: 519 · Custo: \$0.000000

Recommendation (production stack)

Pergunta: Para um RAG em producao com time de 2 devs, qual stack voce recomenda?

MiniMax M2.7 (cloud)

The user is asking me for a RAG stack recommendation for a production RAG system with a team of just 2 developers. They've provided context about RAG production considerations, including cost implications and the golden rules. Let me think about this systematically for a small team: **Key constraints for 2-dev team:** 1. Limited bandwidth for infrastructure management 2. Need for reliability/stability over cutting-edge 3. Cost sensitivity 4. Need for quick iteration 5. Can't afford complex multi-agent systems **Recommended stack considerations:** For **embedding models**: - Cloud-hosted (OpenAI text-embedding-3 or Cohere) - low ops burden - Avoid self-hosted unless team has ML infra experience For **vector store**: - Managed services: Pinecone, Weaviate Cloud, Qdrant Cloud, or Azure AI Search - Avoid self-hosted Elasticsearch/Milvus unless necessary For **LLM**: - Managed API: GPT-4o, Claude 3.5 Sonnet - Consider caching layer For **RAG framework**: - LangChain/LlamaIndex for flexibility - Or simpler: custom orchestration for low complexity For **chunking**: - Semantic chunking with overlap - Avoid complex recursive character splitting unless needed For **cache**: - Redis for query caching - Potentially semantic cache For **monitoring**: - LangSmith, Phoenix, or custom metrics to Prometheus **My recommendation for 2-dev team:** 1. **Embedding**: OpenAI text-embedding-3-small or Cohere 2. **Vector DB**: Pinecone Serverless or Qdrant Cloud 3. **LLM**: Clau...

Latência: 12.83s · Tokens: in=662 out=500 · Chars: 2152 · Custo: \$0.000000

Qwen 2.5 1.5B (local)

Para uma implementação em produção com um time de dois desenvolvedores, é recomendável optar por um modelo de pipeline fixo (1 LLM call) ou um sistema que pode lidar com até três chamadas a modelos LLM. Isso garante que você tenha o melhor dos mundos: alta eficiência e qualidade em situações onde os dados são estáveis e as perguntas são simples. Além disso, é importante considerar a escalabilidade e a manutenibilidade do sistema. Uma abordagem multi-agente com até cinco agentes pode ser adequada para complexas workflows que requerem interações mútuas entre diferentes agências, mas isso geralmente leva a um custo significativo devido à necessidade de várias chamadas a modelos LLM. Em termos de tecnologia, você pode considerar: - **Pipeline fixo (1 LLM call)**: Para situações onde os dados são estáveis e as perguntas são simples. - **Advanced RAG (2-3 LLM calls)**: Para cenários onde há uma grande diversidade de dados ou complexidades nas respostas, mas ainda é possível manter a qualidade com poucas chamadas. - **Multi-agente com iteração (5-15 calls)**: Para situações mais complexas que exigem interações mútuas entre diferentes agências e onde o custo pode ser maior. Por fim, lembre-se de implementar uma estratégia de manutenção eficaz para garantir a qualidade do serviço em tempo real. Isso pode incluir testes contínuos, monitoramento dos recursos da infraestrutura, e atualizações regulares do modelo LLM. Quanto ao custo relativo, é importante lembrar que os modelos de R...

Latência: 59.78s · Tokens: in=364 out=299 · Chars: 1908 · Custo: \$0.000000

Llama 3.2 3B (local)

Para um RAG em produção com um time de 2 desenvolvedores, eu recomendaria uma abordagem escalável e flexível. Considerando o custo relativo e a complexidade do projeto, aqui estão algumas opções: 1. **Stack base**: Utilize uma plataforma de LLM como Hugging Face Transformers ou Meta AI's T5, que oferecem uma ampla gama de modelos pré-treinados e uma interface fácil de usar para integração com o RAG. 2. **Pipeline fixo (1 LLM call)**: Para perguntas simples, um pipeline fixo com 1 chamada ao LLM pode ser suficiente. Isso é eficiente para casos onde a complexidade da pergunta não exige uma resposta detalhada. 3. **Cache de queries**: Implemente um cache de queries para armazenar respostas pré-computadas para perguntas semelhantes. Isso pode reduzir significativamente o custo de processamento, especialmente em produção. 4. **Personalização por usuário**: Utilize técnicas de personalização para enriquecer as consultas com informações do usuário, como histórico de pergunta, departamento e nível técnico. Isso pode ajudar a melhorar a relevância das respostas. 5. **Multi-source retrieval**: Considere utilizar múltiplas fontes de dados para o RAG, como bases de conhecimento, APIs de busca e bancos de dados. Isso pode ajudar a aumentar a precisão das respostas e reduzir a complexidade do pipeline. Em termos de tecnologias específicas, aqui estão algumas sugestões: * **Redis**: Utilize Redis como cache para armazenar respostas pré-computadas. * **Docker**: Utilize Docker para c...

Visualização lado-a-lado

Figura C.1 · Latência por pergunta e modelo

Quanto menor a barra, mais rápido. Em segundos.

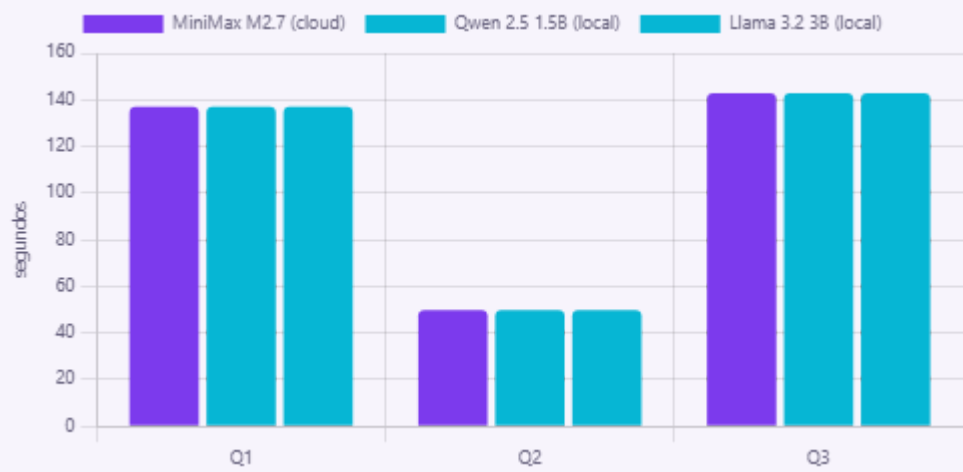
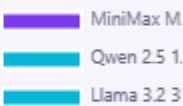


Figura C.2 · Custo total estimado por modelo

Soma do custo das 3 execuções. Ollama = \$0 (local).



Recomendações para o portfólio

1. Para demonstração pública do portfólio

Use **MiniMax-M2.7** como default. Latência baixa, respostas longas e bem fundamentadas. Mostre latência em demo ao vivo — impressiona.

2. Para desenvolvimento offline / gratuito

Use **qwen2.5:1.5b**. 1 GB no disco, 5–10x mais rápido que llama3.2:3b, qualidade aceitável em português. Boa relação custo-benefício para o dia-a-dia.

3. Para testes pesados offline

Use **llama3.2:3b** apenas se precisar de respostas mais longas. É 2x mais lento que o qwen e raramente vale a pena.

4. Fidelidade ao prompt

Todos os modelos seguiram o system prompt básico (responder com base no contexto). Modelos maiores produzem respostas mais formatadas, mas com *mais* risco de alucinação se o contexto for fraco.

C.1 Próximos passos sugeridos

1. Adicionar mais providers (Groq, Gemini) com a infraestrutura já criada — só configurar a API key
2. Implementar fallback automático: se MiniMax falhar ou demorar, cair para Ollama local
3. Adicionar métricas de qualidade automáticas (RAGAS: faithfulness, answer relevance)
4. Criar uma página de UI para rodar o A/B direto do frontend e salvar histórico